LLM applications

with R and ellmer

Hadley Wickham

Chief Scientist, Posit

CC BY SA

1. Programming with LLMs
2. Structured data
3. Non-text inputs
4.

Programming with ellmer

https://ellmer.tidyverse.org/articles/programming.html

Two basic options

```
# You create the chat object
capital ← function(country) {
  chat ← chat_openai(model = "gpt-4.1-mini")
  text ← interpolate("What's the capital of {country}")
  chat$chat(input)
}

# User provides the chat
capital ← function(chat, country) {
  text ← interpolate("What's the capital of {country}")
  chat$chat(input)
}
```

Three challenges with user provided

- Input object is changed (mutability)
- Input object brings state
- User-created chat will stream by default

R objects are usually immutable

```
x ← list(a = 1, b = 2)

f ← function() {
  x$a ← 100
}
f()

# The original x is unchanged
str(x)
```

But ellmer objects are not

```
chat ← chat_openai("Be terse", model = "gpt-4.1-nano")

capital ← function(chat, country) {

  chat$chat(interpolate("What's the capital of {{country}}"))
}
capital(chat, "New Zealand")
capital(chat, "France")

chat
```

Use $clone() to create a copy

```
chat ← chat_openai("Be terse", model = "gpt-4.1-nano")

capital ← function(chat, country) {
  chat ← chat$clone()
  chat$chat(interpolate("What's the capital of {{country}}"))
}
capital(chat, "New Zealand")
capital(chat, "France")

chat
```

Input object brings state

```
chat ← chat_openai("Be terse")
chat$chat("Pretend the capital of New Zealand is Hadleyville.")


capital ← function(chat, country) {
  chat ← chat$clone()
  chat$chat(interpolate("What's the capital of {{country}}"))
}
capital(chat, "New Zealand")
```

So discard all input turns

```
chat ← chat_openai("Be terse")
chat$chat("Pretend the capital of New Zealand is Hadleyville.")

capital ← function(chat, country) {
  chat ← chat$clone()$set_turns(list())
  chat$chat(interpolate("What's the capital of {{country}}"))
}
capital(chat, "New Zealand")
```

Chat objects created at top-level stream by default

```
chat ← chat_openai("Be terse", model = "gpt-4.1-nano")


capital ← function(chat, country) {
  chat ← chat$clone()$set_turns(list())
  chat$chat(interpolate("What's the capital of {{country}}"))
}
nz ← capital(chat, "New Zealand")
```

Fix this by setting echo = "none"

```
chat ← chat_openai("Be terse", model = "gpt-4.1-nano")

capital ← function(chat, country) {
  chat ← chat$clone()
  chat$chat(
    interpolate("What's the capital of {{country}}"),
    echo = "none"
  )
}
x ← capital(chat, "New Zealand")
```

Your turn

- Write a function that returns the name of an R function given a description of it. You supply the chat object.
- Write a function that returns the name of an R function given a description of it. Allow the user to supply a chat object.

Structured data

https://ellmer.tidyverse.org/articles/structured-data.html

LLMs are incredibly useful for unstructured data

```
prompts ← list(
  "I go by Alex. 42 years on this planet and counting.",
  "Pleased to meet you! I'm Jamal, age 27.",
  "They call me Li Wei. Nineteen years young.",
  "Fatima here. Just celebrated my 35th birthday last week.",
  "The name's Robert - 51 years old and proud of it.",
  "Kwame here - just hit the big 5-0 this year."
)

# How would you solve this with R code?
```

This is trivial with an LLM

```
# Simpler models often adequate for extraction
chat ← chat_claude(
  "Extract name and age from my inputs. Just give the values.",
  model = "claude-haiku-4-5-20251001"
)

chat$chat(prompts[[1]])
```

And we could turn this into a function

```
extract_name_age ← function(x) {
  chat ← chat_claude(
    "Extract name and age from my inputs. Just give the values.",
    model = "claude-haiku-4-5-20251001",
    echo = "none"
  )
  chat$chat(x)
}

extract_name_age("I go by Alex. 42 years on this planet and counting.")
# But what do we do with this string?
```

$chat_structured to the rescue!

```
type_person ← type_object(
  name = type_string(),
  age = type_number()
)

chat$chat_structured(prompts[[1]], type = type_person)
```

It’s very common to want to do this en masse

```
# Fast
parallel_chat_structured(chat, prompts, type = type_person)

# Slow but 50% cheaper
batch_chat_structured(
  chat,
  prompts,
  path = "name-age.json",
  type = type_person
)
```

Results are cached in this file, so you can resume in a later session, if needed

Key to structured data are the type_ functions

JSON	$chat_structured	parallel_chat()
`type_string()`	Single string	Character vector
`type_number()`	Single number	Numeric vector
`type_boolean()`	Single T/F	Logical vector
`type_object()`	Named list	Data frame

Your turn

- What happens if you give it data without name or age? Generate a few examples that are missing name or age or both and see what happens.
- (You might need to use convert = FALSE to see the root cause of the problem)
- What argument to type_string() and type_number() might help? (Read the docs). What impact does it make on your examples?
- Do you need any additional prompting?

Non-text inputs

	Image	PDF	Video
OpenAI	✅	✅	
Claude	✅	✅	
Gemini	✅	✅	✅

Providing non-text inputs

```
chat ← chat_claude()

chat$chat(
  content_image_file("holly-mandarich-3p9zaNwUtv8-unsplash.jpg"),
  "Describe this photo"
)
chat$chat("Where in the world do you think it is?")

chat$get_tokens()
```

Other functions

```
content_image_url()
content_image_file()

content_pdf_url()
content_pdf_file()

# For chat_google_gemini()
google_upload()
```

Your turn

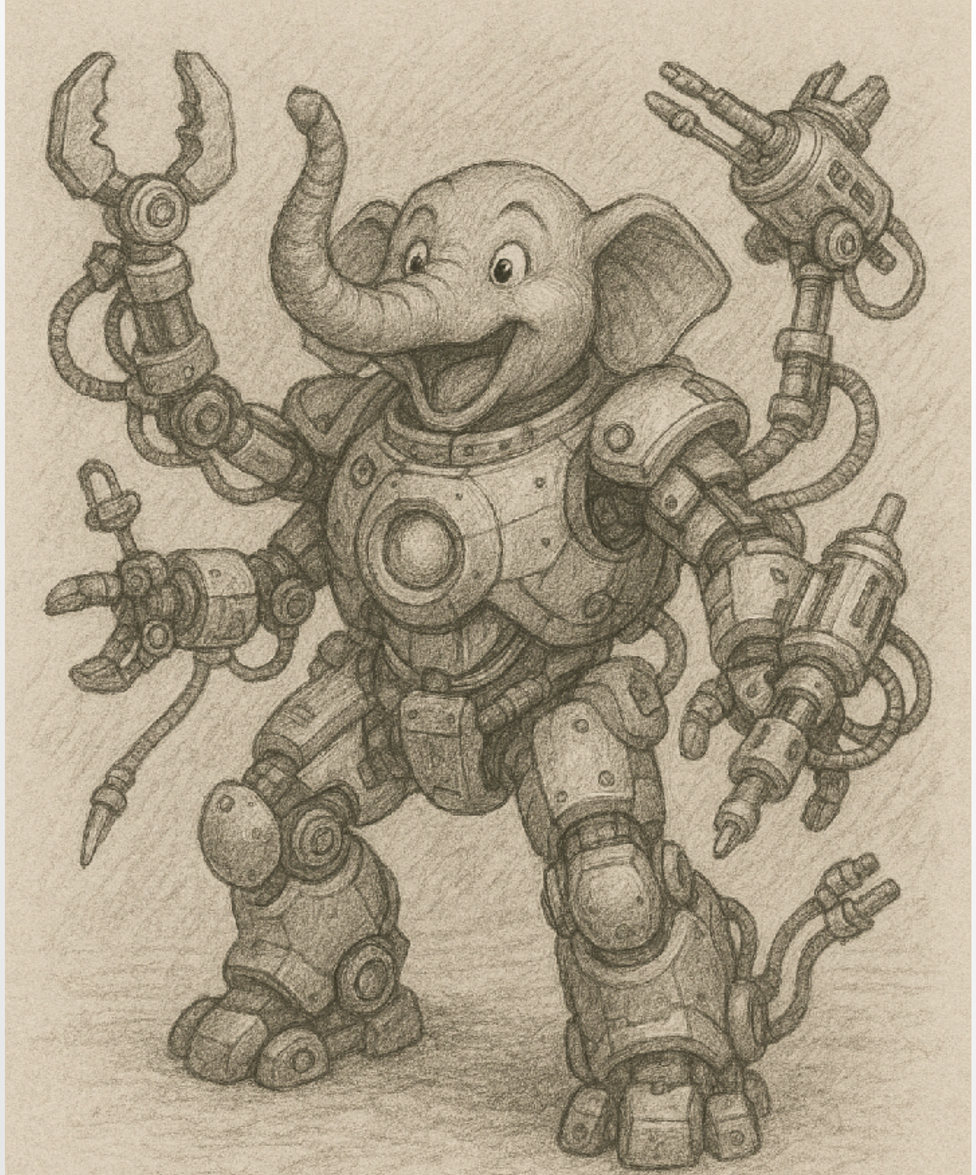
- Write an OCR function. It should take a path to an image and return the text in the image formatted as markdown.
- Write a plot metadata extractor. It should take a path to an image and extract the x axis label, y axis label, and title.

Tool calling

https://ellmer.tidyverse.org/articles/tool-calling.html



LLM



LLM with tool calling

Built-in tools

- Providers now come with a variety of built-in tools. ellmer currently provides access to two: web search and web fetch.
- You can guess what web search does. Web fetch allows the LLM to read pages on the internet.
- See claude_tool_web_search(), openai_tool_web_search(), google_tool_web_search() etc for details.
- Very useful for “research” type projects

Your turn

- Write a recipe extraction function: it should take a url to a recipe you found on the internet and return a list containing a character vector of ingredients and a character vector of steps.
- (Hints: you'll need to use a web fetch tool and $chat_structured().)

Tool calling

- Can also provide the LLM with your own tools
- A tool is just another name for a function
- Useful for:
  - Things LLMs aren't good at (e.g. basic maths)
  - Providing current information (e.g. today’s date)
  - Looking up extra info (e.g. R documentation)
  - Doing something on your behalf (this makes it an **agent**!)

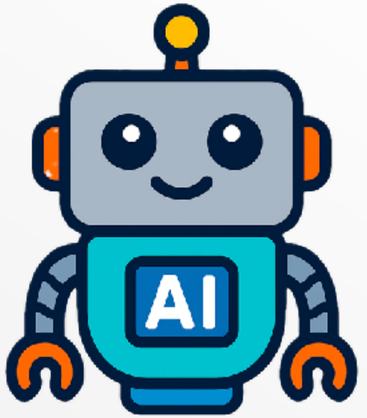
LLMs don’t know what day it is. But you can provide a tool:

```
chat ← chat_openai("You're a terse assistant")
chat$chat("What's today's date?")
#> Today’s date is June 13, 2024

chat$register_tool(tool(
  function() Sys.Date(),
  name = "today",
  description = "Gets the current date"
))
chat$chat("What's today's date?")
#> Let me get today's date for you.
#> ○ [tool call] today()
#> ● #> "2025-11-23"
#> Today’s date is November 23, 2025.
```



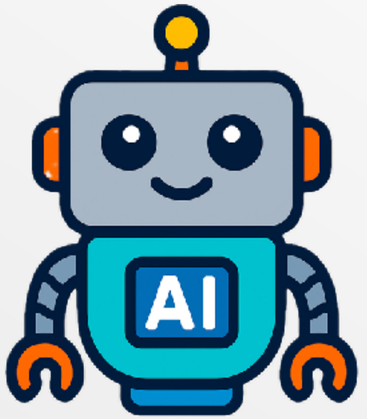
What day is it today?



Please call today()

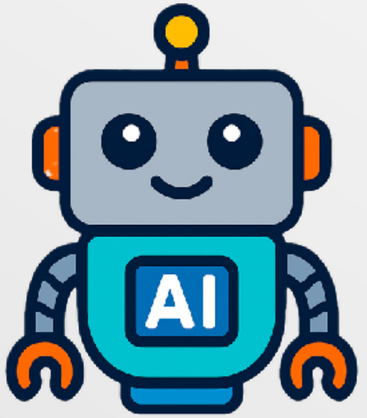


2025-11-19



Today is **November 19, 2025**.





How does this work?

```
<Chat Anthropic/claude-3-7-sonnet-latest turns=7 tokens=875/71 $0.00>
── system [0] ──────────────────────────────────────────────────────────────────
You're a terse assistant
── user [19] ───────────────────────────────────────────────────────────────────
What's today's date?
── assistant [13] ──────────────────────────────────────────────────────────────
Today is May 20, 2023.
── user [365] ──────────────────────────────────────────────────────────────────
What's today's date?
── assistant [44] ──────────────────────────────────────────────────────────────
I'll check today's date for you.
[tool request (toolu_01VR1eRhnpFeJxW5bnsKFe9r)]: today()
── user [18] ───────────────────────────────────────────────────────────────────
[tool result  (toolu_01VR1eRhnpFeJxW5bnsKFe9r)]: "2025-06-13"
── assistant [14] ──────────────────────────────────────────────────────────────
Today is June 13, 2025.
```

Your turn

- Write a function that returns the correct age of a famous person.
- (Gemini seems to have some challenges with this so you'll need to do this in three steps: first ask for the birth date, then register the tool, then ask for age.)

NB: now you can do “multiagent” AI

```
summarise_text ← function(text) {
  chat ← chat_openai(model = "gpt-4.1-nano", echo = FALSE)
  chat$chat(c("Summarise the following text into 3 bullets", "", text))
}

chat ← chat_anthropic()
chat$register_tool(
  summarise_text,
  "Summarise text",
  text = type_string("Text to summarise"),
)
```

Your turn!

Project ideas

Create a custom prompt to solve a common coding problem. See https://simonpcouch.github.io/chores/ for some examples.

Use tool calling to give the LLM additional info.

Extract structured data from unstructured text, PDFs, images, video, ...

If you're familiar with shiny, you could use https://github.com/jcheng5/shinychat to make your own chatbot. If you're not, ask https://jcheng.shinyapps.io/ellmer-assistant/ to make you an app.

If you get stuck, try https://jcheng.shinyapps.io/ellmer-assistant/